

```
1  /* =====
2     SkyLedger – Airline Inventory Management
3     In-memory data layer + UI bindings.
4     Cabins are always tracked in this fixed order:
5     economy, premium, business, first
6     ===== */
7
8  const CABINS = [
9    { key: "economy", label: "Economy" },
10   { key: "premium", label: "Premium Economy" },
11   { key: "business", label: "Business" },
12   { key: "first", label: "First" },
13 ];
14
15 /* -----
16     DATA STORE
17     ----- */
18 const store = {
19   aircraft: [],
20   routes: [],
21   flights: [],
22   fares: [],
23   _ids: { aircraft: 1, route: 1, flight: 1, fare: 1 },
24 };
25
26 function nextId(kind) {
27   return kind.charAt(0).toUpperCase() + kind.slice(1) + "-" + String(store._ids[kind]++).padStart(3, "0");
28 }
29
30 /* -----
31     SEED DATA – gives the console something to show on load
32     ----- */
33 function seed() {
34   const a1 = addAircraft({
35     tail: "VT-SLA", type: "Airbus A321neo",
36     economy: 150, premium: 24, business: 16, first: 0,
37     maintenance: "2026-08-14", maintenanceNote: "A-check",
38   });
39   const a2 = addAircraft({
40     tail: "VT-SLB", type: "Boeing 787-9",
41     economy: 210, premium: 35, business: 30, first: 8,
42     maintenance: "2026-09-02", maintenanceNote: "C-check",
43   });
44   const a3 = addAircraft({
45     tail: "VT-SLC", type: "Airbus A320",
46     economy: 168, premium: 0, business: 12, first: 0,
47     maintenance: "2026-08-05", maintenanceNote: "Line check",
48   });
49
50   const r1 = addRoute({ source: "DEL", destination: "BOM", stops: [] });
51   const r2 = addRoute({ source: "BLR", destination: "LHR", stops: ["DEL"] });
52   const r3 = addRoute({ source: "MAA", destination: "SIN", stops: [] });
53
54   const f1 = addFlight({ flightNumber: "SL 204", routeId: r1.id, aircraftId: a1.id, date: "2026-07-30", time: "06:15" });
55   const f2 = addFlight({ flightNumber: "SL 811", routeId: r2.id, aircraftId: a2.id, date: "2026-07-30", time: "23:40" });
56   const f3 = addFlight({ flightNumber: "SL 552", routeId: r3.id, aircraftId: a3.id, date: "2026-07-31", time: "14:05" });
57
58   bookSeats(f1.id, "economy", 132);
```

```
59     bookSeats(f1.id, "premium", 20);
60     bookSeats(f1.id, "business", 16);
61     bookSeats(f2.id, "economy", 90);
62     bookSeats(f2.id, "business", 30);
63     bookSeats(f3.id, "economy", 168);
64     bookSeats(f3.id, "business", 4);
65
66     addFare({ flightId: f1.id, cabin: "economy", fareName: "Y – Full Flex", price: 8200, allocated: 40 });
67     addFare({ flightId: f1.id, cabin: "economy", fareName: "Q – Saver", price: 4300, allocated: 110 });
68     addFare({ flightId: f1.id, cabin: "business", fareName: "C – Business Flex", price: 21000, allocated: 1
69     6 });
69     addFare({ flightId: f2.id, cabin: "economy", fareName: "Y – Full Flex", price: 46000, allocated: 60 });
70     addFare({ flightId: f2.id, cabin: "business", fareName: "C – Business Flex", price: 145000, allocated: 3
71     0 });
72 }
73
74 /* -----
75    AIRCRAFT MANAGEMENT
76    ----- */
77 function addAircraft(data) {
78     const record = {
79         id: nextId("aircraft"),
80         tail: data.tail,
81         type: data.type,
82         config: {
83             economy: Number(data.economy) || 0,
84             premium: Number(data.premium) || 0,
85             business: Number(data.business) || 0,
86             first: Number(data.first) || 0,
87         },
88         maintenance: data.maintenance || "",
89         maintenanceNote: data.maintenanceNote || "",
90     };
91     store.aircraft.push(record);
92     return record;
93 }
94
95 function capacityOf(aircraft) {
96     return CABINS.reduce((sum, c) => sum + aircraft.config[c.key], 0);
97 }
98
99 function removeAircraft(id) {
100     const inUse = store.flights.some(f => f.aircraftId === id && f.status !== "cancelled");
101     if (inUse) {
102         return { ok: false, message: "Cannot remove aircraft – it is assigned to an active flight." };
103     }
104     store.aircraft = store.aircraft.filter(a => a.id !== id);
105     return { ok: true };
106 }
107
108 /* -----
109    ROUTE MANAGEMENT
110    ----- */
111 function addRoute(data) {
112     const record = {
113         id: nextId("route"),
114         source: data.source.toUpperCase(),
115         destination: data.destination.toUpperCase(),
116         stops: (data.stops || []).map(s => s.trim().toUpperCase()).filter(Boolean),
117     };
118     store.routes.push(record);
119     return record;
```

```
119 }
120
121 function removeRoute(id) {
122     const inUse = store.flights.some(f => f.routeId === id && f.status !== "cancelled");
123     if (inUse) {
124         return { ok: false, message: "Cannot remove route – flights are still scheduled on it." };
125     }
126     store.routes = store.routes.filter(r => r.id !== id);
127     return { ok: true };
128 }
129
130 /* -----
131    FLIGHT MANAGEMENT + SEAT INVENTORY
132    ----- */
133 function buildInventoryFromAircraft(aircraft) {
134     const inv = {};
135     CABINS.forEach(c => {
136         inv[c.key] = { total: aircraft.config[c.key], booked: 0 };
137     });
138     return inv;
139 }
140
141 function addFlight(data) {
142     const aircraft = store.aircraft.find(a => a.id === data.aircraftId);
143     const record = {
144         id: nextId("flight"),
145         flightNumber: data.flightNumber,
146         routeId: data.routeId,
147         aircraftId: data.aircraftId,
148         date: data.date,
149         time: data.time,
150         status: "scheduled",
151         inventory: buildInventoryFromAircraft(aircraft),
152     };
153     store.flights.push(record);
154     return record;
155 }
156
157 function updateFlight(id, changes) {
158     const flight = store.flights.find(f => f.id === id);
159     if (!flight) return { ok: false, message: "Flight not found." };
160     if (changes.flightNumber) flight.flightNumber = changes.flightNumber;
161     if (changes.date) flight.date = changes.date;
162     if (changes.time) flight.time = changes.time;
163     if (changes.routeId) flight.routeId = changes.routeId;
164     return { ok: true };
165 }
166
167 function assignAircraft(flightId, aircraftId) {
168     const flight = store.flights.find(f => f.id === flightId);
169     const aircraft = store.aircraft.find(a => a.id === aircraftId);
170     if (!flight || !aircraft) return { ok: false, message: "Flight or aircraft not found." };
171
172     const bookedByCabin = {};
173     CABINS.forEach(c => bookedByCabin[c.key] = flight.inventory[c.key].booked);
174     const overbooked = CABINS.some(c => bookedByCabin[c.key] > aircraft.config[c.key]);
175     if (overbooked) {
176         return { ok: false, message: "New aircraft has fewer seats than are already booked in one or more c
abins." };
177     }
178
179     flight.aircraftId = aircraftId;
```

```
180     CABINS.forEach(c => {
181         flight.inventory[c.key].total = aircraft.config[c.key];
182     });
183     return { ok: true };
184 }
185
186 function cancelFlight(id) {
187     const flight = store.flights.find(f => f.id === id);
188     if (!flight) return { ok: false, message: "Flight not found." };
189     flight.status = "cancelled";
190     return { ok: true };
191 }
192
193 function removeFlightPermanently(id) {
194     store.flights = store.flights.filter(f => f.id !== id);
195     store.fares = store.fares.filter(fr => fr.flightId !== id);
196 }
197
198 /* -----
199     AVAILABILITY CHECKING + INVENTORY SYNC
200     (booking / cancellation instantly updates seat counts)
201     ----- */
202 const OVERBOOK_ALLOWANCE = 0.03; // 3% controlled overbooking ceiling
203
204 function availability(flightId, cabin) {
205     const flight = store.flights.find(f => f.id === flightId);
206     if (!flight) return null;
207     const inv = flight.inventory[cabin];
208     return {
209         total: inv.total,
210         booked: inv.booked,
211         available: inv.total - inv.booked,
212     };
213 }
214
215 function bookSeats(flightId, cabin, count, allowOverbook = false) {
216     const flight = store.flights.find(f => f.id === flightId);
217     if (!flight) return { ok: false, message: "Flight not found." };
218     if (flight.status === "cancelled") return { ok: false, message: "Cannot book seats on a cancelled flight." };
219
220     const inv = flight.inventory[cabin];
221     const ceiling = allowOverbook ? Math.floor(inv.total * (1 + OVERBOOK_ALLOWANCE)) : inv.total;
222
223     if (inv.booked + count > ceiling) {
224         return {
225             ok: false,
226             message: `Only ${ceiling - inv.booked} seat(s) available in ${cabin} on ${flight.flightNumber} (prevented overbooking).`,
227         };
228     }
229
230     inv.booked += count;
231     return { ok: true, remaining: inv.total - inv.booked };
232 }
233
234 function releaseSeats(flightId, cabin, count) {
235     const flight = store.flights.find(f => f.id === flightId);
236     if (!flight) return { ok: false, message: "Flight not found." };
237     const inv = flight.inventory[cabin];
238     const releaseCount = Math.min(count, inv.booked);
239     inv.booked -= releaseCount;
```

```
240     return { ok: true, released: releaseCount, remaining: inv.total - inv.booked };
241   }
242
243   /* -----
244     FARE INVENTORY (fare classes allocated within a cabin)
245     ----- */
246   function addFare(data) {
247     const record = {
248       id: nextId("fare"),
249       flightId: data.flightId,
250       cabin: data.cabin,
251       fareName: data.fareName,
252       price: Number(data.price),
253       allocated: Number(data.allocated),
254     };
255     store.fares.push(record);
256     return record;
257   }
258
259   function removeFare(id) {
260     store.fares = store.fares.filter(f => f.id !== id);
261   }
262
263   /* =====
264     UI LAYER
265     ===== */
266   const $ = (sel, root = document) => root.querySelector(sel);
267   const $$ = (sel, root = document) => Array.from(root.querySelectorAll(sel));
268
269   function fmtMoney(n) {
270     return "■" + Number(n).toLocaleString("en-IN");
271   }
272
273   function routeLabel(routeId) {
274     const r = store.routes.find(x => x.id === routeId);
275     if (!r) return "-";
276     const stops = r.stops.length ? ` (via ${r.stops.join(", ")} )` : "";
277     return `${r.source} → ${r.destination}${stops}`;
278   }
279
280   function aircraftLabel(aircraftId) {
281     const a = store.aircraft.find(x => x.id === aircraftId);
282     return a ? `${a.tail} · ${a.type}` : "-";
283   }
284
285   function flightLabel(flightId) {
286     const f = store.flights.find(x => x.id === flightId);
287     return f ? `${f.flightNumber} - ${routeLabel(f.routeId)}` : "-";
288   }
289
290   function cabinLabel(key) {
291     return (CABINS.find(c => c.key === key) || {}).label || key;
292   }
293
294   function loadClass(pct) {
295     if (pct >= 100) return "is-full";
296     if (pct >= 85) return "is-warn";
297     return "";
298   }
299
300   function showToast(message, isError = false) {
301     const el = $("#toast");
```

```

302     el.textContent = message;
303     el.classList.toggle("is-error", isError);
304     el.classList.add("is-visible");
305     clearTimeout(showToast._t);
306     showToast._t = setTimeout(() => el.classList.remove("is-visible"), 3200);
307 }
308
309 /* ----- navigation ----- */
310 const VIEW_META = {
311     dashboard: ["Operations Dashboard", "Live snapshot of the network"],
312     flights: ["Flight Management", "Add, update, cancel, and reassign flights"],
313     aircraft: ["Aircraft Management", "Fleet, cabin configuration, and maintenance"],
314     routes: ["Route Management", "Source, destination, and intermediate stops"],
315     fares: ["Fare Inventory", "Fare classes allocated within each cabin"],
316     availability: ["Availability & Sync", "Real-time seat availability and inventory sync"],
317 };
318
319 function setView(name) {
320     $$(".rail-nav__item").forEach(b => b.classList.toggle("is-active", b.dataset.view === name));
321     $$(".view").forEach(v => v.classList.toggle("is-active", v.id === "view-" + name));
322     $("#viewTitle").textContent = VIEW_META[name][0];
323     $("#viewSubtitle").textContent = VIEW_META[name][1];
324 }
325
326 $$(".rail-nav__item").forEach(btn => {
327     btn.addEventListener("click", () => setView(btn.dataset.view));
328 });
329
330 /* ----- clock ----- */
331 function tickClock() {
332     const now = new Date();
333     $("#railClock").textContent = now.toLocaleTimeString("en-GB");
334 }
335 setInterval(tickClock, 1000);
336 tickClock();
337
338 /* ----- select population ----- */
339 function fillSelect(select, items, current) {
340     select.innerHTML = items.map(([value, label]) =>
341         `<option value="${value}" ${value === current ? "selected" : ""}>${label}</option>`
342     ).join("");
343 }
344
345 function refreshSelects() {
346     const routeOpts = store.routes.map(r => [r.id, `${r.id} · ${routeLabel(r.id)}`]);
347     const aircraftOpts = store.aircraft.map(a => [a.id, `${a.id} · ${a.tail} (${a.type})`]);
348     const flightOpts = store.flights
349         .filter(f => f.status !== "cancelled")
350         .map(f => [f.id, `${f.flightNumber} · ${routeLabel(f.routeId)} · ${f.date}`]);
351
352     fillSelect($("#flightRouteSelect"), routeOpts);
353     fillSelect($("#flightAircraftSelect"), aircraftOpts);
354     fillSelect($("#fareFlightSelect"), flightOpts);
355     fillSelect($("#bookingFlightSelect"), flightOpts);
356 }
357
358 /* ----- rendering ----- */
359 function render() {
360     refreshSelects();
361     renderDashboard();
362     renderFlights();
363     renderAircraft();

```

```

364     renderRoutes();
365     renderFares();
366     renderAvailability();
367 }
368
369 function renderDashboard() {
370     const activeFlights = store.flights.filter(f => f.status !== "cancelled");
371     $("#statFlights").textContent = activeFlights.length;
372     $("#statAircraft").textContent = store.aircraft.length;
373     $("#statRoutes").textContent = store.routes.length;
374
375     let totalBooked = 0, totalSeats = 0;
376     activeFlights.forEach(f => CABINS.forEach(c => {
377         totalBooked += f.inventory[c.key].booked;
378         totalSeats += f.inventory[c.key].total;
379     }));
380     $("#statBooked").textContent = totalBooked;
381     const pct = totalSeats ? Math.round((totalBooked / totalSeats) * 100) : 0;
382     $("#statBookedPct").textContent = `${pct}% load factor`;
383
384     const tbody = $("#dashboardBoard tbody");
385     if (!activeFlights.length) {
386         tbody.innerHTML = `<tr><td class="table-empty" colspan="9">No active flights yet. Add one from the
Flights tab.</td></tr>`;
387     } else {
388         tbody.innerHTML = activeFlights.map(f => {
389             const cabinCells = CABINS.map(c => {
390                 const inv = f.inventory[c.key];
391                 const avail = inv.total - inv.booked;
392                 const pct = inv.total ? Math.round((inv.booked / inv.total) * 100) : 0;
393                 return `<td><span class="seat-chip"><b>${avail}</b> open / ${inv.total}
394                     <span class="bar ${loadClass(pct)}"><span style="width:${Math.min(pct,100)}%"></span></span>
395                     </span></td>`;
396             }).join("");
397             return `<tr>
398                 <td class="mono">${f.flightNumber}</td>
399                 <td>${routeLabel(f.routeId)}</td>
400                 <td class="mono">${f.date} ${f.time}</td>
401                 <td>${aircraftLabel(f.aircraftId)}</td>
402                 ${cabinCells}
403                 <td><span class="badge badge--${f.status}">${f.status}</span></td>
404             </tr>`;
405         }).join("");
406     }
407
408     const watch = $("#maintenanceWatch");
409     const upcoming = [...store.aircraft]
410         .filter(a => a.maintenance)
411         .sort((a, b) => a.maintenance.localeCompare(b.maintenance));
412     watch.innerHTML = upcoming.length ? upcoming.map(a => `
413         <li>
414             <div>
415                 <div class="tail">${a.tail}</div>
416                 <div class="note">${a.maintenanceNote || "Scheduled hold"}</div>
417             </div>
418             <div class="date">${a.maintenance}</div>
419         </li>
420     `).join("") : `<li class="note">No maintenance scheduled.</li>`;
421 }
422
423 function renderFlights() {
424     const tbody = $("#flightsTable tbody");

```

```

425   if (!store.flights.length) {
426     tbody.innerHTML = `<tr><td class="table-empty" colspan="7">No flights yet.</td></tr>`;
427     return;
428   }
429   tbody.innerHTML = store.flights.map(f => {
430     const totalBooked = CABINS.reduce((s, c) => s + f.inventory[c.key].booked, 0);
431     const totalSeats = CABINS.reduce((s, c) => s + f.inventory[c.key].total, 0);
432     return `<tr>
433       <td class="mono">${f.flightNumber}<br><span style="color:var(--text-dim)">${f.id}</span></td>
434       <td>${routeLabel(f.routeId)}</td>
435       <td>${aircraftLabel(f.aircraftId)}</td>
436       <td class="mono">${f.date} ${f.time}</td>
437       <td class="mono">${totalBooked} / ${totalSeats}</td>
438       <td><span class="badge badge--${f.status}">${f.status}</span></td>
439       <td>
440         <div class="row-actions">
441           <button class="btn btn--ghost btn--small" data-cancel-flight="${f.id}" ${f.status === "cancel
442 led" ? "disabled" : ""}>Cancel</button>
443           <button class="btn btn--ghost btn--small" data-delete-flight="${f.id}">Delete</button>
444         </div>
445       </td>
446     </tr>`;
447   }).join("");
448   $$("#[data-cancel-flight]").forEach(btn => btn.addEventListener("click", () => {
449     cancelFlight(btn.dataset.cancelFlight);
450     showToast("Flight cancelled. Inventory locked.");
451     render();
452   }));
453   $$("#[data-delete-flight]").forEach(btn => btn.addEventListener("click", () => {
454     removeFlightPermanently(btn.dataset.deleteFlight);
455     showToast("Flight record removed.");
456     render();
457   }));
458 }
459
460 function renderAircraft() {
461   const tbody = $("#aircraftTable tbody");
462   if (!store.aircraft.length) {
463     tbody.innerHTML = `<tr><td class="table-empty" colspan="6">No aircraft registered.</td></tr>`;
464     return;
465   }
466   tbody.innerHTML = store.aircraft.map(a => {
467     const config = CABINS.map(c => `${c.label.split(" ")[0]}:${a.config[c.key]}`).join(" ");
468     return `<tr>
469       <td class="mono">${a.tail}</td>
470       <td>${a.type}</td>
471       <td class="mono">${capacityOf(a)}</td>
472       <td class="mono">${config}</td>
473       <td class="mono">${a.maintenance || "-"}${a.maintenanceNote ? `<br><span style="color:var(--text-
474 dim)">${a.maintenanceNote}</span>` : ""}</td>
475       <td><button class="btn btn--ghost btn--small" data-delete-aircraft="${a.id}">Remove</button></td>
476     </tr>`;
477   }).join("");
478   $$("#[data-delete-aircraft]").forEach(btn => btn.addEventListener("click", () => {
479     const res = removeAircraft(btn.dataset.deleteAircraft);
480     if (!res.ok) return showToast(res.message, true);
481     showToast("Aircraft removed from fleet.");
482     render();
483   }));
484 }

```



```
485
486 function renderRoutes() {
487   const tbody = $("#routesTable tbody");
488   if (!store.routes.length) {
489     tbody.innerHTML = `|  |  |  |  |  |  |  |  |  |  |  |  |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| No routes defined.</td></tr>`; 490     return; 491   } 492   tbody.innerHTML = store.routes.map(r => { 493     const flightsOnRoute = store.flights.filter(f => f.routeId === r.id && f.status !== "cancelled").length; 494     return `|  |  |  |  |  |  | | --- | --- | --- | --- | --- | --- | | 495       <td class="mono">${r.id}</td> 496       <td class="mono">${r.source}</td> 497       <td class="mono">${r.destination}</td> 498       <td>${r.stops.length ? r.stops.join(", ") : "Direct"}</td> 499       <td class="mono">${flightsOnRoute}</td> 500       <td><button class="btn btn--ghost btn--small" data-delete-route="${r.id}">Remove</button></td> 501     </tr>`; 502   }).join(""); 503 504   $("[data-delete-route]").forEach(btn => btn.addEventListener("click", () => { 505     const res = removeRoute(btn.dataset.deleteRoute); 506     if (!res.ok) return showToast(res.message, true); 507     showToast("Route removed."); 508     render(); 509   })); 510 } 511 512 function renderFares() { 513   const tbody = $("#faresTable tbody"); 514   if (!store.fares.length) { 515     tbody.innerHTML = `|  |  |  |  |  |  | | --- | --- | --- | --- | --- | --- | | No fare classes allocated yet.</td></tr>`; 516     return; 517   } 518   tbody.innerHTML = store.fares.map(fr => ` 519     <tr> 520       <td>${flightLabel(fr.flightId)}</td> 521       <td>${cabinLabel(fr.cabin)}</td> 522       <td>${fr.fareName}</td> 523       <td class="mono">${fmtMoney(fr.price)}</td> 524       <td class="mono">${fr.allocated}</td> 525       <td><button class="btn btn--ghost btn--small" data-delete-fare="${fr.id}">Remove</button></td> 526     </tr> 527   `).join(""); 528 529   $("[data-delete-fare]").forEach(btn => btn.addEventListener("click", () => { 530     removeFare(btn.dataset.deleteFare); 531     showToast("Fare class removed."); 532     render(); 533   })); 534 } 535 536 function renderAvailability() { 537   const tbody = $("#availabilityTable tbody"); 538   const rows = []; 539   store.flights.filter(f => f.status !== "cancelled").forEach(f => { 540     CABINS.forEach(c => { 541       const inv = f.inventory[c.key]; 542       if (inv.total === 0) return; 543       const avail = inv.total - inv.booked; 544       const pct = Math.round((inv.booked / inv.total) * 100); 545       rows.push(` | | | | | | | | | | | |

```

```

546         <td class="mono">${f.flightNumber}</td>
547         <td>${c.label}</td>
548         <td class="mono">${inv.total}</td>
549         <td class="mono">${inv.booked}</td>
550         <td class="mono">${avail}</td>
551         <td><span class="bar ${loadClass(pct)}" style="width:80px"><span style="width:${Math.min(pct,10
0)}%"></span></span></td>
552     </tr>`);
553 });
554 });
555 tbody.innerHTML = rows.length ? rows.join("") : `<tr><td class="table-empty" colspan="6">No inventory
to display.</td></tr>`;
556 }
557
558 /* ----- form: schedule flight ----- */
559 $("#flightForm").addEventListener("submit", e => {
560     e.preventDefault();
561     const fd = new FormData(e.target);
562     addFlight({
563         flightNumber: fd.get("flightNumber"),
564         routeId: fd.get("routeId"),
565         aircraftId: fd.get("aircraftId"),
566         date: fd.get("date"),
567         time: fd.get("time"),
568     });
569     showToast("Flight scheduled and inventory initialised.");
570     e.target.reset();
571     render();
572 });
573
574 /* ----- form: register aircraft ----- */
575 $("#aircraftForm").addEventListener("submit", e => {
576     e.preventDefault();
577     const fd = new FormData(e.target);
578     addAircraft({
579         tail: fd.get("tail"),
580         type: fd.get("type"),
581         economy: fd.get("economy"),
582         premium: fd.get("premium"),
583         business: fd.get("business"),
584         first: fd.get("first"),
585         maintenance: fd.get("maintenance"),
586         maintenanceNote: fd.get("maintenanceNote"),
587     });
588     showToast("Aircraft added to fleet.");
589     e.target.reset();
590     render();
591 });
592
593 /* ----- form: define route ----- */
594 $("#routeForm").addEventListener("submit", e => {
595     e.preventDefault();
596     const fd = new FormData(e.target);
597     const stopsRaw = fd.get("stops") || "";
598     addRoute({
599         source: fd.get("source"),
600         destination: fd.get("destination"),
601         stops: stopsRaw.split(",").map(s => s.trim()).filter(Boolean),
602     });
603     showToast("Route added to network.");
604     e.target.reset();
605     render();

```

```

606 });
607
608 /* ----- form: allocate fare ----- */
609 $("#fareForm").addEventListener("submit", e => {
610     e.preventDefault();
611     const fd = new FormData(e.target);
612     const flightId = fd.get("flightId");
613     const cabin = fd.get("cabin");
614     const allocated = Number(fd.get("allocated"));
615
616     const avail = availability(flightId, cabin);
617     if (!avail) return showToast("Select a flight first.", true);
618
619     const alreadyAllocated = store.fares
620         .filter(f => f.flightId === flightId && f.cabin === cabin)
621         .reduce((s, f) => s + f.allocated, 0);
622
623     if (alreadyAllocated + allocated > avail.total) {
624         showToast(`Allocation exceeds ${cabinLabel(cabin)} capacity (${avail.total} seats total, ${alreadyA
llocated} already allocated).`, true);
625         return;
626     }
627
628     addFare({
629         flightId,
630         cabin,
631         fareName: fd.get("fareName"),
632         price: fd.get("price"),
633         allocated,
634     });
635     showToast("Fare class allocated.");
636     e.target.reset();
637     render();
638 });
639
640 /* ----- form: booking / release (availability sync) ----- */
641 function logRow(message, ok) {
642     const log = $("#bookingLog");
643     const row = document.createElement("div");
644     row.className = "log_row " + (ok ? "is-ok" : "is-error");
645     const stamp = new Date().toLocaleTimeString("en-GB");
646     row.textContent = `[${stamp}] ${message}`;
647     log.prepend(row);
648 }
649
650 $("#bookingForm").addEventListener("submit", e => {
651     e.preventDefault();
652     const action = e.submitter ? e.submitter.dataset.action : "book";
653     const fd = new FormData(e.target);
654     const flightId = fd.get("flightId");
655     const cabin = fd.get("cabin");
656     const count = Number(fd.get("count"));
657     const allowOverbook = $("#allowOverbook").checked;
658
659     if (!flightId) return showToast("Add a flight first.", true);
660
661     if (action === "book") {
662         const res = bookSeats(flightId, cabin, count, allowOverbook);
663         if (!res.ok) { logRow(res.message, false); showToast(res.message, true); }
664         else { logRow(`Booked ${count} × ${cabinLabel(cabin)} on ${flightLabel(flightId).split(" -")[0]} - $
{res.remaining} left.`, true); showToast("Booking confirmed. Inventory synced."); }
665     } else {

```

```
666     const res = releaseSeats(flightId, cabin, count);
667     logRow(`Released ${res.released} × ${cabinLabel(cabin)} on ${flightLabel(flightId).split(" -")[0]}
- ${res.remaining} left.`, true);
668     showToast("Booking cancelled. Inventory synced.");
669   }
670   render();
671 });
672
673 /* -----
674    INIT
675    ----- */
676 seed();
677 render();
678 setView("dashboard");
679
```